INDEX

Name: LOHESH M CS23532 Roll No: 240701292

Std: Subject: COMPUTER NETWORKS School/College:

Sl. No.	Date	Title	Page No	Remarks
1.	20/7/26	Study of Various network commands used in LINUX and WINDOWS		
2.	30/7/26	Different types of network cables.		
3.	30/7/26	PACKET TRACER tool and user Interface.		
4.	27/8/26	Setting up a Local Area Network (LAN)		
5.	1/9/26	Packet Capture tool: WIRESHARK		
6.	7/9/26	Error Detection & correction - Hamming code		
7.	28/7/26	Error Control at Data Link layer		
8.	28/9/26	Flow control at data link layer		
9.	28/9/26	NMAP to Discover live hosts Using NMAP Scans (ARP, ICMP, TCP/UDP) on TryHackMe platform.		
10.	27/9/26	Implementation of Subnetting in CISCO Packet tracer Simulation		

Sl. No.	Date	Title	Page No	Remarks
11.	29/09/26	Interconnecting with routers in CISCO packet Tracer Simulation		
12.	30/9/26	Routing at network layer.		
13.	1/10/26	End to End communication at Transport layer.		
14.	7/10/26	Implement your own Ping Program		
15.	9/10/26	Packet Sniffing using RAW socket		

Completed

Ex.No:1 STUDY OF VARIOUS NETWORK COMMAND USED IN LINUX AND WINDOWS

AIM:

To study the various network commands used in Linux and Windows.

COMMANDS:

1) arp -a

Show IP address of your computer along with IP Address and MAC address of your router.

o/p

Interface 192.168.56.1 --- 0x7

Internet Address	Physical Address	Type
192.168.56.255	ff-ff-ff-ff-ff-ff	Static
244.00.22.0	01-00-5e-002	Static
23-9-255-25-250	0-1-00-5e-ff-ff	Static

2) hostname:

Display name of your computer

o/p : naveed.

3) ipconfig /all: Display detailed configuration information about TCP/IP connection and type of ethernet adapter in the system.

o/p Window IP Configuration

Host name . . . . . : naveed

primary DNS suffix

route type

4) Nb Stat -a : Display protocol statistic and current TCP/IP connection using

o/p

NBTSTAT [[-a Remote Name][A IP addres

5) netstat - Display a variety of statistics a a computer active TCP/IP connecti

o/p

Active connections

protocol	local address	Foreign address	Stat
Tcp	10-76-9-6-60:49175	10-76-96-163:domain	Tim
Tcp	10-76-96-60:52289	142-250-183:http	Sta
Tcp	10-76-96-60:56954	10-76-96-163:domain	Tim

6) nslookup : Display DNS detail such as IP address of the particular computer

o/p

Server : unknown
Address : 10.76.96.163

7) pathping : traces the route to destination addre the number of seconds lost of each router along the way

o/p pathping [-g host-list] [-n m as] [-i address]

Linux Networking commands

1) ip : Help from setting up new system and assigning IP to trouble shooting existing system.

2) ipconfig : The ipconfig is staple in many system administrator admins tool for configuring trouble shooting.

3) mtr : MTR is free program with CLI that are a network diagnostic and trouble shooting tool.

4) tcpdump : The tcpdump command is designed for capturing displaying packets

5) Ping : ping is tool that verifies IP - Local connectivity to another TCP/IP computer by sending Internet control message protocol (ICMP) Echo request message protocol (ICMP) Echo request message.

6) IP config :

flag : 73 UP : Loop back, RUNNING mtu 65536

inet 127.0.0.1 netmask : 255.0.0.0

inet 6 : : : prefix len 128 scope host

7) mtr - o/p

my traceroute [v0.87]

local host local domain (::1)

Key Help Display mode Restart Sta

Host Packets Loss% Lo

::1 0-0%

8) tcpdump -i

Listening on enp 250 of 401

IP 693-243-207-34.bc

http > navneet-5658, seq=5929, ack

to

9) ping

ping 8-8-8-8

64 bytes from 8-8-8-8 (pcon

seq=1 ttl=

time = 37.4ms

RESULT: Thus the various network commands used in Linux and Windows are executed successfully.

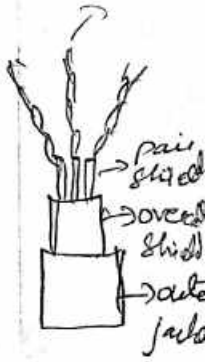
DIFFERENT TYPES OF NETWORK CABLES

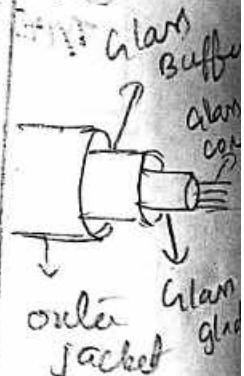
AIM:

To study and understand different types of network cables.

DIFFERENT TYPES OF NETWORK CABLES:

1. Unshielded Twisted pair (UTP) cables.
2. Shielded Twisted Pair (STP) cables
3. Coaxial cables
4. Fibre optic cables

CABLE TYPE	CATEGORY	MAX DATA TRANSMISSION	ADV/ DISADV	APPLICATION/ USE	IMAGE
UTP	Category 3	10 bps	Adv ⇒ Cheaper in cost ⇒ Easy to install as they have a smaller overall diameter Disadv ⇒ More prone to (EMI) & noise	10 Base -T Ethernet	
	Category 5	Upto 100 Mbps		Fast Ethernet Gigabit Ethernet	
	Category 5e	1 Gbps		Fast Ethernet Gigabit Ethernet	
STP	Category 6, 6a	10 Gbps	Adv ⇒ shielded ⇒ faster than UTP ⇒ less susceptible to noise and interference Disadv ⇒ Expensive ⇒ Greater installation effort	Gigabit Ethernet, 10G Ethernet (55m) widely used in data centres	
SSTP	Category 7	10 Gbps		Gigabit Ethernet 10G Ethernet (100m)	

CABLE TYPE	CATEGORY	MAX DATA TRANSMISS	ADV / DISADV	APPLICATION / USE	IMAGE
Coaxial Cable	RG-6 RG-59 RG-11	10-100 Mbps	Adv ⇒ High bandwidth ⇒ Immune to interference ⇒ low loss bandwidth ⇒ Versatile Disadv ⇒ limited distance ⇒ Cost ⇒ Size is bulky	Speed of signal 500m TV network High speed internet connections	
Fibre optics Cable	Singlemode Multi mode	100 Gbps	Adv ⇒ High Speed ⇒ High BW ⇒ high security ⇒ Long Distance Disadv ⇒ Expensive ⇒ Requires skilled installer	Maximum distance of fibre optics Cable is around 100 meter	

RESULT:

Thus the types of network cables have been studied and understood.

Ex. NO. 3

PACKET TRACER TOOL AND USER INTERFACE

AIM

To study the packet tracer tool Installation and user Interface overview.

To understand environment of CISCO PACKET TRACER to design simple network.

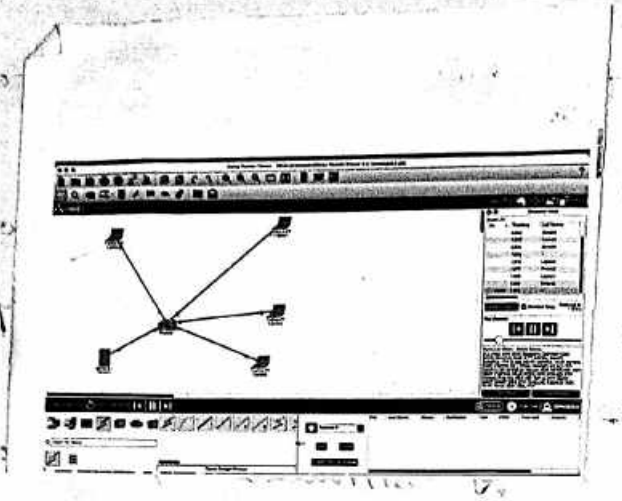
INTRODUCTION:

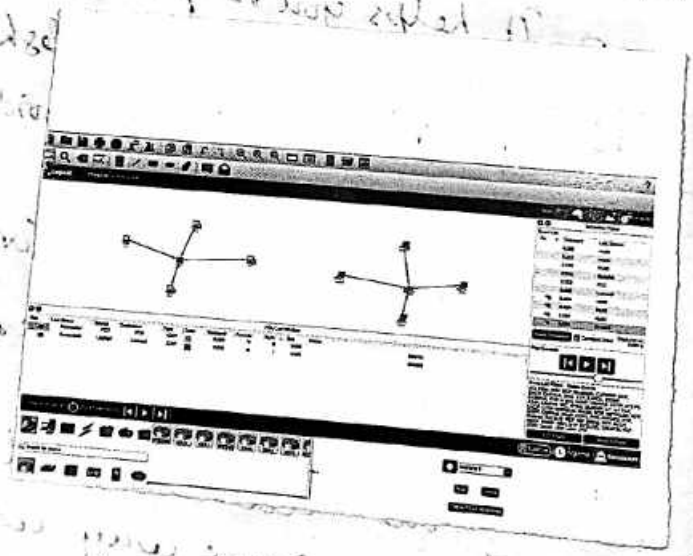
A simulator, as the name suggests, simulates network devices and its environment. Packet Tracer is an exciting network design, simulation and modelling tool.

1. It allows you to model complex systems without the need for dedicated equipment.
2. It helps you to practice your network configuration and troubleshooting skills via computer or an Android or IOS based mobile device
3. It is available for both the Linux and windows desktop environment
4. Protocols in Packet Tracer are coded to work and behave in the same way as they would on real hardware.

INSTALLING PACKET TRACER

To download Packet Tracer, go to http://www.netcad.com and log in with your CISCO networking Academy Credentials; then click on the Packet Tracer graphic and download the package appropriate for your operating System.





WINDOWS:

Installation in windows is pretty simple and straightforward; the setup comes in a single file named Packettracer_setup_6.0.1.exe. Open this file to begin the setup wizard, accept the license agreement, choose a location and start the installation.

LINUX:

Linux users with an Ubuntu/Debian distribution should download the file for Ubuntu and those using Fedora/Redhat/CentOS must download the file for Fedora. Grant executable permission to this file by using chmod, and execute it to begin the installation.

chmod +x PacketTracer601_i386_installer-rpm-bin
./PacketTracer601_i386_installer-rpm-bin

USER INTERFACE OVERVIEW:

The layout of Packet Tracer is divided into several components. The components of the Packet Tracer interface are as follows; match the numbering and explanations.

1. Menubar - this is a common menu found in all software applications; it is used to open, save, print, change preferences and so on.
2. Main toolbar - this bar provides shortcut icons to menu options that are commonly accessed, such as open, save, zoom, undo, and redo.

3. Logical / Physical Workspace tabs - These tabs allow you to toggle between the logical and physical work areas.

4. Workspace: This is the area where topologies are created and simulation are displayed.

5. Common toolbar :- This toolbar provides controls for manipulating topologies, such as select, move layout, place note, delete, inspect resize shape and add simple/complex PDU.

6. Real time / Simulation tabs - These tabs are used to toggle between the real and simulation modes.

7. User created packet box - User can create highly customized packets to test their topology from this area. and results are displayed on a list.

ANALYSE THE BEHAVIOUR OF NETWORK DEVICES USING CISCO PACKET TRACER SIMULATOR

1. From the network component box, click and drag and drop the below components.
   a. 4 Generic PCs and one HUB
   b. 4 Generic PCs and one switch

2. Click on connections
   a - Click on copper - Straight through cable.
   b - Select one the PC and connect it to HUB using the cable. The link LED should glow in green, indicating that the link is up. Similarly connect remaining 3PCs to the HUB

C. Similarly connect 4 PCs to the switch using copper straight-through cable

3. Click on the PCs connected to hubs. Go to the Desktop tab, click on IP configuration and enter an IP address and subnet mask. Here the default gateway and DNS server information is not needed. As there are only two end devices in the network

4. Observe the flow of PDU from source PC to destination PC by selecting the Realtime mode of simulation

5- Repeat the steps #3 to 5 for the PCs connected to the switch.

6. Observe how the HUB and switch are forwarded the PDU and observe and conclude the behaviour of switch and hub

RESULT:

The PACKET TRACER Tool has been installed and studied the behaviour of switch and HUB in simple network.

EX.NO:4

SETTING UP A LOCAL AREA NETWORK (LAN)

AIM:

To design, setup and configure a Local Area Network (LAN) connecting four computers through a network switch, assign static IP address to all devices. Verify connectivity using the ping command and demonstrate resource/data sharing across the network.

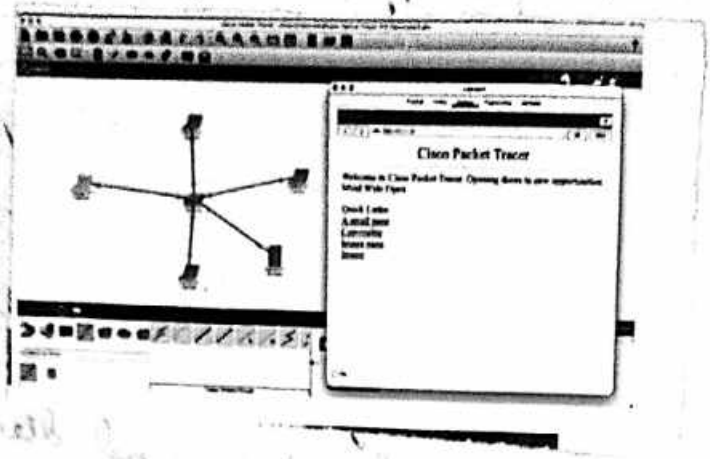
EQUIPMENT REQUIRED

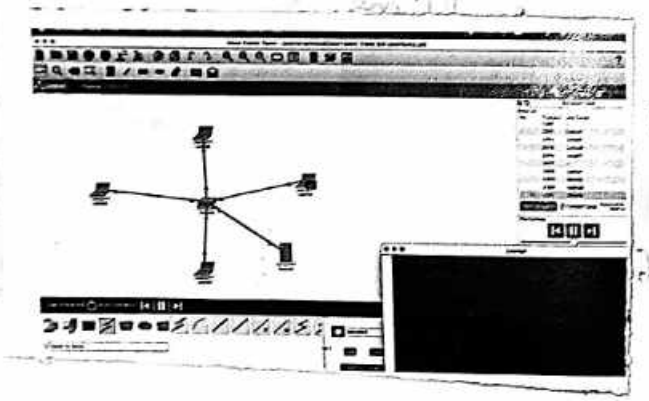
S.NO	Equipment	Quantity	Specification
1.	Personal computer (PC)	4	Ethernet (NIC) enabled.
2.	Network switch	1	8/16/24 port unmanaged/managed
3.	Ethernet cables (cat5e/cat6)	4	Straight-through, RJ-45 connector

THEORY:

A Local Area Network (LAN) is a network that connects computers and devices within a limited geographical area, such as a home, office or laboratory, allowing them to share resources such as files, printers and internet connection.

Star topology is used in this experiment where all end devices (PC) are individually connected to a central n/w switch. The switch operates at layer 2 of OSI model and forwards data frames between connected devices based on their MAC addresses. Every device on a LAN must be assigned an IP address within the same network range to be able to communicate with one another.


Cisco Packet Tracer



IP Address Configuration Table.

Device	IP Address	Subnet mask	Connected to
PC0	10.1.1.1	255.0.0.0	Switch Port Fa0/1
PC1	10.1.1.2	255.0.0.0	Switch Port Fa0/2
PC2	10.1.1.3	255.0.0.0	Switch Port Fa0/3
PC3	10.1.1.4	255.0.0.0	Switch Port Fa0/4
Switch (Management)	10.1.1.5	255.0.0.0	VLAN 1 Interface

IMPLEMENTATION IN CISCO PACKET TRACER

Step 1: Place the Devices

1. Open Cisco Packet Tracer.
2. From the device category panel, select End devices and drag 4 PCs onto the workspace.
3. Select Switches and drag 1 switch onto the workspace.

Step 2 Cable the Devices

4. Click Connections - Select Copper Straight-Through cable.
5. Click each PC, then click the switch and select an available FastEthernet port to complete each connection.
6. Repeat for all 4 PCs. Green link light confirms a successful physical connection.

Step 3 Assign IP addresses to PCs

7. Click a PC → Desktop tab → IP Configuration
8. Enter the static IP Address and Subnet Mask as per the IP table. For each PC

Step 4 Configure the Switch Management IP (optional)

9. Click the switch → CLI tab.
10. Enter the following commands to assign management IP enable

Configure terminal
Interface Vlan1
ip address 10.1.1.5 255.0.0.0
no shutdown
exit

Step 5 Verify connectivity using Ping.

11. click a PC → Desktop tab → command prompt
12. use the ping command to test connectivity to the other PCs. eg ping 10.1.1.2.
13. Successful "Reply from" message. Confirm connectivity

Step 6 Demostrate Data Sharing (Server simulation)

14. Add a server device and connect it to a switch
15. Assign it an IP address (eg. 10.1.1.10, subnet mask 255.0.0.0) via its Desktop → IP address Configuration tab.
16. Open the server's services (config) tab and enable the HTTP service.
17. From another PC, open the web browser (Desktop tab) and enter the server's IP address to load the page, confirming successful data transfer across the LAN.

Step 7 Observe Packet flow in Simulation Mode.

18. Switch to simulation mode
19. Send a ping or web request between two devices and use the play (capture controls to step through the packet as it travels hop-by-hop through the switch, observing MAC address based forwarding.

OBSERVATION: All 4 PCs were successfully connected to the switch in a star topology and assigned unique static IP address within the same subnet. Ping test between all devices returned successful replies, confirming end-to-end connectivity. The shared server was successfully accessed from other computers in the n/w, confirming that the LAN was functioning correctly.

Ex. NO 5

PACKET CAPTURE TOOL: WIRESHARK

AIM:

To study the Packet capture tool: Wireshark.

To study the network traffic, analyse the traffic and flow on the network.

INTRODUCTION:

Wireshark is a network analysis tool formerly known as Ethereal, captures packets in real time and display them in human-readable format.

Wireshark includes filters, color coding, and other features like network traffic and inspect individual packets.

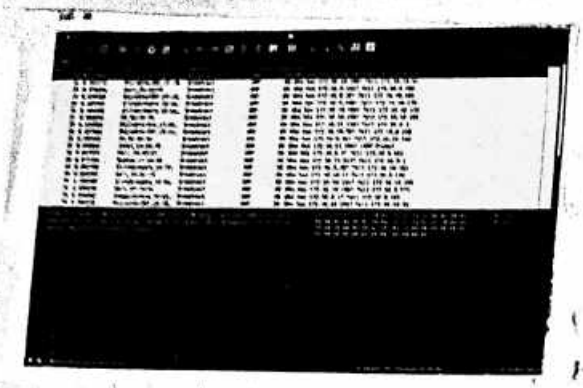
Wireshark used for

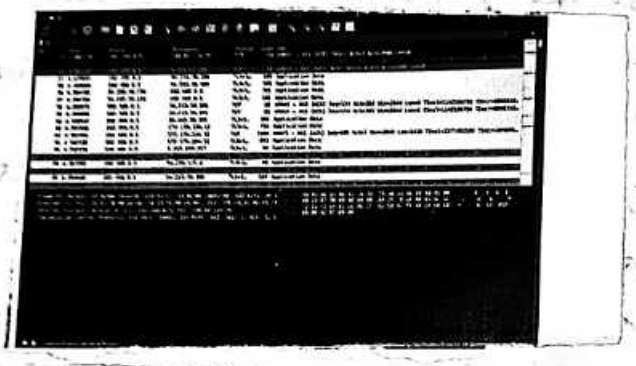
* Network administrators: troubleshoot network problems
* Network security engineers: examine security problems
* Developers: debug protocol implementation
* Learn network protocol internals

CAPTURING AND ANALYSING PACKETS USING WIRESHARK TOOL:

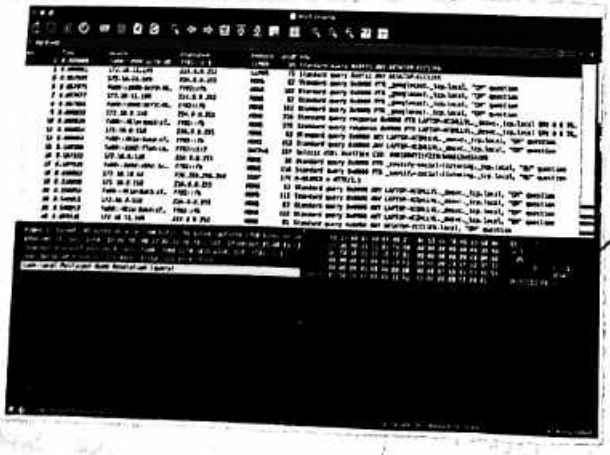
To filter, capture, view, packets in Wireshark tool.

Capture 100 packets from the Ethernet IEEE 802.3 LAN interface and save it.









[redacted]

PROCEDURE:

1. Select Local Area Connection in wireshark.
2. Goto capture option
3. Select stop Capture automatically after 100 packets
4. Then click Start capture.
5. Save the packets

1. Create a filter to display only TCP/UDP packets, inspect the packets and provide the flow graph.

PROCEDURE:

1. Select LAN in wireshark.
2. Goto capture option
3. Select stop capture automatically after 100 packets
4. Then click start capture.
5. Search TCP packets in search bar.
6. To see flow graph statistics Flow graph
7. Save the Packets

2. Create a filter to display ARP packets and inspect the packets.

PROCEDURE:

1. Go to capture option
2. select stop capture automatically after 100 packets
3. Then click start capture
4. Search ARP packet in search bar.
5. Save the packets

3. Create a filter to display only DNS packets and provide the flow graph.

PROCEDURE:

1. Goto capture option
2. Select stop capture automatically after 100 packets.
3. Then click start capture.
4. Search DNS packets in search bar.
5. To see flow graph click Statistics.

4. Create a filter to display only HTTP packets and inspect the packets

PROCEDURE:

1. Select LAN in wireshark.
2. Go to capture option
3. Select stop capture automatically after 100 packets
4. Then click start capture
5. Search HTTP packets in search bar.

5. Create a filter to display only IP/ICMP packets and inspect the packets

PROCEDURE:

1. Select LAN connection in wireshark.
2. Goto capture option
3. Select stop capture automatically after 100 packet
4. Then click start capture.
5. Search ICMP/IP packets in seach bar

6. Create a filter to display only DHCP packets and inspect the packets

PROCEDURE:

1. Select Local Area connection in wireshark
2. Go to capture option
3. Select capture automatically after 100 packet
4. Then click Start capture
5. Search DHCP packet in search bar.
6. Save the packet.

7.

RESULT:

The Packet capture tool - Wireshark has been installed and studied. The network traffic and traffic flow of the network have been observed.

Ex.No:6

ERROR DETECTION & CORRECTION - HAMMING CODE

AIM:

Write a program to implement error detection and correction using HAMMING code concept. Make a test run to input data stream and verify error correction feature.

ERROR CORRECTION AT DATA LINK LAYER:

Hamming code is a set of error-correction codes that can be used to detect and correct the errors that can occur when the data is transmitted from the sender to the receiver. It is a technique developed by R.W. Hamming for error correction.

CREATE SENDER PROGRAM WITH BELOW FEATURES

1. Input to sender file should be a text of any length. Program should convert the text to binary.
2. Apply hamming code concept on the binary data and add redundant bits to it.
3. Save this output in a file called channel.

CREATE A RECEIVER PROGRAM WITH BELOW FEATURES

1. Receiver program should read the input from channel file.
2. Apply hamming code on the binary data to check for errors.
3. If there is an error, display the position of the error
4. Else remove the redundant bits and convert the binary data to ASCII and display the output

SENDER PROGRAM:

```
def text_to_binary(text):
    return ' '.join(format(ord(c), '08b') for c in text)

def hamming_encode(data):
    encoded_data = ''
    for i in range(0, len(data), 4):
        d1 = int(data[i])
        d2 = int(data[i+1])
        d3 = int(data[i+2])
        d4 = int(data[i+3])

        p1 = d1 ^ d2 ^ d4
        p2 = d1 ^ d3 ^ d4
        p4 = d2 ^ d3 ^ d4

        encoded_data += (
            str(p1) + str(p2) + str(d1) + str(p4) +
            str(d2) + str(d3) + str(d4)
        )
    return encoded_data
text = input("Enter the text to send: ")
binary_data = text_to_binary(text)
print("\n original text:")
print(text)
print("\n Binary Data:")
print(binary_data)
encoded_data = hamming_encode(binary_data)
print("\n Hamming Encoded data:")
print(encoded_data)
with open("channel.txt", "w") as file:
    file.write(encoded_data)
```

Print("the channel.txt file represent the communication

Input

Enter the text to send : ~~hotkesh~~ Hello

Convert Lokesh to Binary

Character 8 bit binary

H 01101000

e 01100101

l 01001100 01001100

o 01101111

complete binary data is;

01101000 01100101 01001100 01001100 01101111

0110 1000 0110 0101 0100 1100 0100 1100 ....

Data bits

d1 d2 d3 d4

Hamming code:

P1 P2 d1 P4 d2 d3 d4

where

$P1 = d1 \oplus d2 \oplus d4$

$P2 = d1 \oplus d3 \oplus d4$

$P3 = d2 \oplus d3 \oplus d4$

The final ~~encoded~~ stream is saved as channel.txt.

Receiver Program:

```
def hamming_decode(received):
    decoded_data = ""
    error_found = False

    for block_no in range(0, len(received), 7):
        block = received[block_no: block_no+7]
        if len(block) != 7:
            continue
        b = [int(x) for x in block]
        error_position = s1*1 + s2*2 + s4*4
        if error_position != 0:
            error_found = True
            actual_position = block_no + error_position
            print(
                f"Error found at bit position "
                f"{actual_position}"
            )
            print(
                f"Error is in block bit position "
                f"{error_position}"
            )
            b[error_position-1] ^= 1
            print("Error corrected")
        d1 = b[2]
        d2 = b[4]
        d3 = b[5]
        d4 = b[6]
        decoded_data += str(d1)+str(d2)+str(d3)
                        +str(d4)
    return decoded_data, error_found
```

```
try:
    with open("channel.txt", "r") as file:
        received_data = file.read().strip()

except FileNotFoundError:
    print("Error: channel.txt file not found.")
    exit()

print("\nData received from channel:")
print(received_data)

decoded_binary, error_found = hamming_decode(received_data)

if not error_found:
    print("\nNo error detected in the received data.")
else:
    print("\nError detected and corrected successfully.")

text = ""
for i in range(0, len(decoded_binary), 8):
    byte = decoded_binary[i:i+8]
    if len(byte)==8:
        decimal_value = int(byte, 2)
        text += chr(decimal_value)
print("\nDecoded Binary Data:")
print(decoded_binary)
print("\nReceived ASCII text:")
print(text)
```

Output:

ASCII binary

h = 01101000
e = 01100101
l = 01101100
l = 01101100
o = 01101111

Result: Thus the Python program has been written for sender & receiver side to check error & correct it using Hamming code

```
import sys
def get_redundant_bits(m):
    for r in range(m+1):
        if (2**r) >= m+r+1:
            return r
def sender_program(text_data):
    binary_data = "".join(format(ord(char), '08b') for char in text_data)
    m = len(binary_data)
    r = get_redundant_bits(m)
    total_len = m+r
    data_rev = binary_data[::-1]
    code = ['0'] * (total_len+1)
    j = 0
    for i in range(1, total_len+1):
        if (i & (i-1)) != 0:
            code[i] = data_rev[j]
            j += 1

    for i in range(r):
        parity_pos = 2**i
        parity_val = 0
        for j in range(1, total_len+1):
            if (j & parity_pos) != 0:
                if j != parity_pos and code[j] ==
                    parity_val ^= 1
                code[parity_pos] = str(parity_val
```

```
codeword = "".join(code[1:])[::-1]
return codeword, m

def receiver_program(received_code, total_data_bits):
    m = total_data_bits
    r = get_redundant_bits(m)
    total_len = len(received_code)
    code = ['0'] + list(received_code[::-1])
    error_position = 0
    for i in range(r):
        parity_pos = 2 ** i
        parity_val = 0
        for j in range(1, total_len + 1):
            if (j & parity_pos) != 0:
                if code[j] == '1':
                    parity_val ^= 1
        if parity_val != 0:
            error_position += parity_pos
    if error_position == 0:
        print("[Receiver]: Transmission received
              successfully! No errors detected.")
    else:
        print(f"[Receiver]: Error identified at bit
        position {error position} from the right side.")
        code[error_position] = '0' if code[error_position] == '1' else '1'
        print("[Receiver]: Successfully corrected the
              corrupt bit.")
    original_data = []
```

```
    for i in range (total_len, 0, -1):
        if (i & (i-1)) != 0:
            original_data.append (code[i])
    corrected_codeword = "".join (code[1:])[::-1]
    decoded_binary = "".join (original_data)
    decoded_text = ""
    for i in range (0, len (decoded_binary), 8):
        byte = decoded_binary[i:i+8]
        if len (byte) == 8:
            decoded_text += chr (int (byte, 2))
    return corrected_codeword, decoded_text
if __name__ == "__main__":
    while True:
        print ("\n Select an option:")
        print ("1. Sender mode (Input text, generate and
                                    Save clean codeword)")
        print ("2. Receiver mode (Read and process codeword
                                    from file)")
        print ("3. Corrupt codeword (Manually flip a bit
                                    inside the file)")
        print ("4. Exit")
        choice = input ("Enter choice (1, 2, 3, or 4):").strip()
        if choice == "1":
            input_message = input ("Enter string (characters):")
            print (f"\n --- SENDER ---")
            print (f"Generated codeword : {encoded_codeword}")
            with open ("codeword-text", "w") as f:
                f.write (f"{total_bits}\n{encoded_
                                    codeword}")
```

```
print("[Sender]: codeword successfully saved to
                'codeword.txt'")

elif choice == "2":
    try:
        with open("codeword.txt", "r") as f:
            lines = f.read().splitlines()
            total_bits = int(lines[0])
            saved_codeword = lines[1]

    except (FileNotFoundError, IndexError):
        print("[Error]: 'codeword.txt' is missing or
                corrupt. Please option 1 first.")

        continue
    print(f"\n--- RECEIVER ---")
    print(f"Reading codeword: {saved_codeword}")
    fixed_signal, parsed_message = receiver_program(saved_
                                    codeword, total_bits)
    print(f"corrected codeword: {fixed_signal}")
    print(f"\nFinal decoded text: {parsed_message}")

elif choice == "3":
    try:
        with open("codeword.txt", "r") as f:
            print(f"current codeword: {saved codeword}")

    try:
        pos = int(input(f"Enter bit position to flip (1 to
                {len(saved_codeword)}, counting from
                right): "))

elif choice == "4"
    print("Existing prg...")
else: print("Invalid selection. please choose 1,2,3, or 4")
```

Program

```
import os
import sys
import time
def read_sender_buffer():
    """ Reads frames currently waiting in the Sender buffer """
    if not os.path.exists("Sender_Buffer.txt"):
        return []
    frames = []
    with open("Sender_Buffer.txt", "r") as f:
        for line in f:
            line = line.strip()
            if line:
                #format: frame no, data
                parts = line.split(",")
                if len(parts) == 2:
                    frames.append((int(parts[0]), parts[1]))
    return frames
def write_sender_buffer(frames):
    """ overwrites the Sender buffer with a new
            window of frames """
    with open("Sender_Buffer.txt", "w") as f:
        for frame in frames:
            f.write(f"{frame no}, {data}\n")
def read_receiver_buffer():
    """ Reads the feedback (ACK/NAK and
                piggybacked data) from the buffer """
    if not os.path.exists("Receiver_Buffer.txt"):
        return None
    with open("Receiver_Buffer.txt", "r") as f:
        line = f.readline().strip()
```

Ex. No. 7.

FLOW CONTROL IN DATA LINK LAYER - SLIDING WINDOW PROTOCOL

AIM:

Write a program to implement flow control at data link layer using Sliding Window Protocol. Simulate the flow of frames from one node to another.

Program should achieve at least below given requirements. Make it a bidirectional program wherein receiver is sending its data frames with acknowledgement (Piggybacking).

CREATE A SENDER PROGRAM WITH FOLLOWING FEATURES

1. Input window size from the user
2. Input a text message from the user
3. Consider 1 character per frame
4. Create a frame with following fields
   [Frame no., DATA]
5. Send the frames
6. Wait for the acknowledgement from the Receiver
7. Reader a file called Receive_Buffer
8. Check ACK. field for the Acknowledgement number
9. If the acknowledgement no. is as expected. Send new set of frames accordingly. Else if NACK is received, resend the frames accordingly

CREATE A RECEIVER FILE WITH FOLLOWING FEATURES

1. Reader a file called Sender_Buffer
2. Check the frame no.
3. If the frame no. are as expected write the

format: type, Ack NO, PIGGYBACK-DATA

```
def Sender node():
    Window_size = int(input("Enter Window size (N):
    message = input("Enter Text message to send:

    while base < total frames:
        1. prepare & send current window
        current-window-frames = [ ]
        end = min(base+window_size, total frames)
        2. Induce Propagation/processing delay
        print("[Sender]: waiting for Receiver feedback...")
        time.sleep(2)
        3. Read feedback from Receiver Buffer
        feedback = read_receiver_buffer()

        # Process ACK/NACK
        if feedback['type'] == "Ack":
            expected ack = base + len(current_window_frames)
            if feedback['ack no'] == expected_ack:

        elif feedback['type'] == "NACK":
            print("[Error]: Invalid target numb...")

        if choice == "1":
            Sender_node()
        elif choice == "2":
            receiver_node()
        elif choice == "3":
            manual_corrupt_node()
        elif choice == "4":
            print("Buffer...")
```

appropriate ACK no. in the Receiver_Buffer file. Else write NACK no. in the Receiver_Buffer file.

RESULT:

Thus the program has been written for flow control in data link layer using Sliding window protocol and executed successfully

EX NO8

NMAP TO DISCOVER LIVE HOSTS USING Nmap SCA
CARP, ICMP, TCP/UDP) ON THE TRYHACKME PLATFORM

AIM:

To Discover Live hosts using Nmap scans (ARP, ICMP, TCP/UDP) on the TryHackMe platform

Room link: http://tryhackme.com/room/nmap01

INTRODUCTION:

When targeting a network, we need an efficient tool to handle repetitive task. This tool should help us to find out which systems are active and what services are running on those systems. The tool is Nmap.

This room is the first of four in this Nmap Series.

- Nmap Live Host Discovery.
- Nmap Basic port Scans
- Nmap Advanced port scans
- Nmap post port scan.

This room explains the steps that Nmap carries out to discover the system that are online before port-scanning the different approaches that used to discover live hosts.

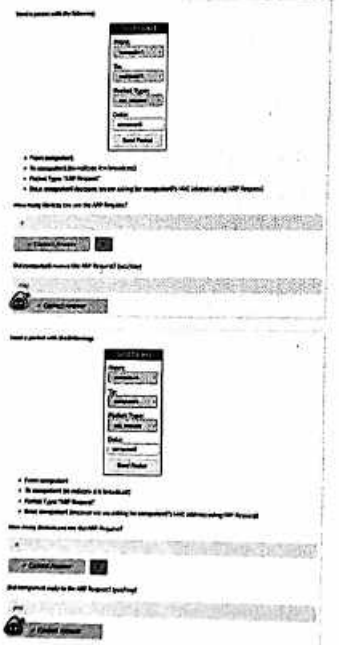
ARP scan : This scan uses ARP request to discover live host

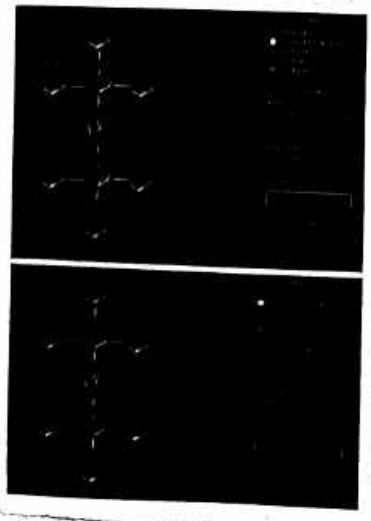
ICMP scan : This scan requests to identify live hosts.

TCP/UDP ping scan: This scan sends packets to TCP ports and UDP ports to determine live hosts.

We also used two scanners : arp-scan and nmap scan.

1. Enumerate Targets
2. Discover live hosts
3 Reverse DNS lookup
4 Scan ports
5. Detect versions
6. Detect OS
7. Trace route
8. Scripts
9. Write output





Subnetwork:

A network segment is a group of computers linked through a shared medium like an Ethernet switch or WiFi access point. Each subnetwork has its own IP address

- There are two types of subnets

/16 subnet: These have a subnet mask of 255.255.0.0 and accomodate 65,535 host

/24 subnet: These have a subnet mask of 255.255.255.0 and support 250 host.

Enumerating Targets:

Before delving into the details of scanning techniques, it is essential to define the targets we want to scan.

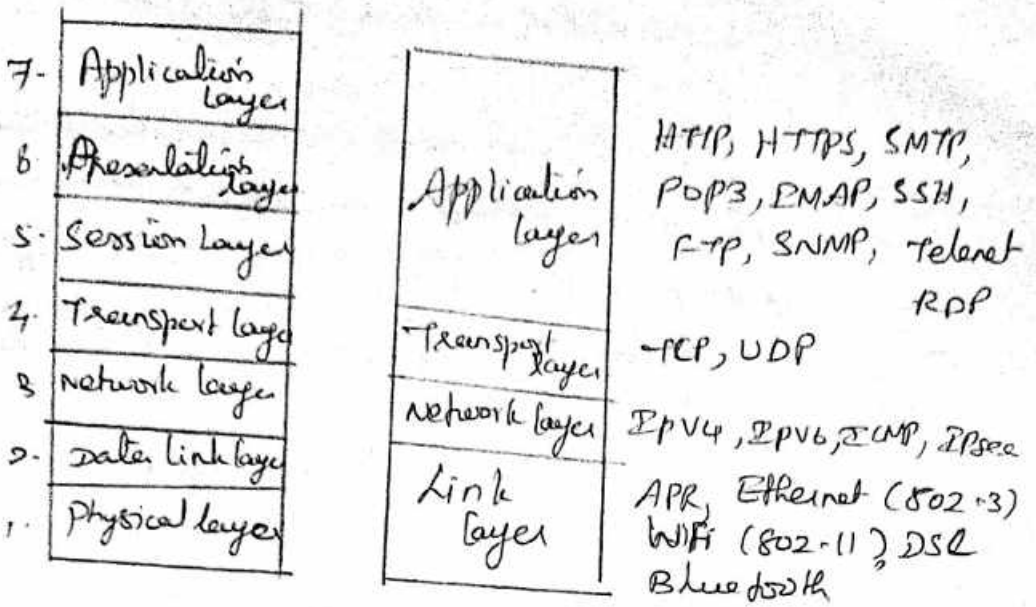
Targets can be specified in 3 ways:

1. List: You can provide a list of IP addresses or host names
2. Range: Specify a range
3. Subnet: can define subnet.

Discovering live hosts

TCP/IP layers:

- ARP from link layer
- ICMP from network layer
- TCP from Transport layer
- UDP from Transport layer



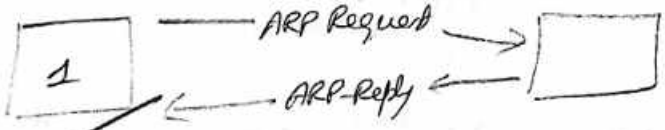
ARP : It is used to request a computer 'MAC' address with a specific IP address.

ICMP : has various types (Type 8 & Type 0).

TCP & UDP : Efficient ~~resp~~ target response checkup when ICMP echo is blocked.

NMAP Host Discovery Using ARP. : when a privileged user tries to scan on a local network.

nmap -PR -sn TARGET



Nmap uses ARP Request

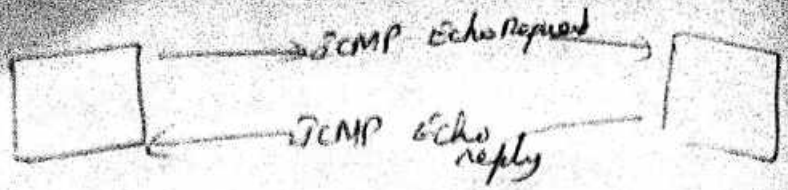
Host is live.

- From computer 1.
- To computer 1
- packet type : " ARP Request "

NMAP Host Discovery using ICMP:

When the privileged user scans outside the network. Nmap uses ICMP echo request, TCP ACK, TCP SYN and ICMP timestamp request.

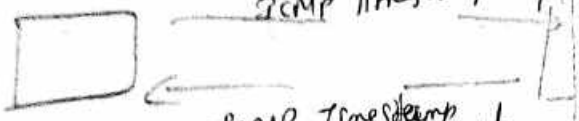
nmap -PE -sn TARGET



An ICMP echo s can u by sending a
ICMP echo request and expect
reply within an ICMP ec

online
PE - sn specifies →

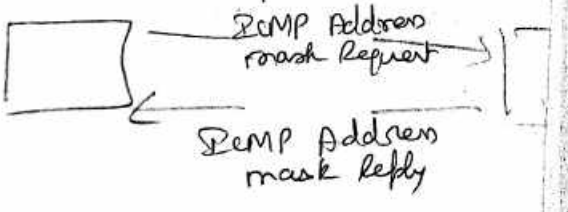
nmap -PP -sn TAR
ICMP Timestamp Requ



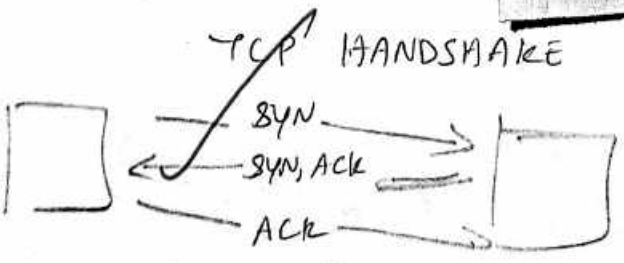
Nmap uses timestamp
whether it will get a Times

Similarly Nmap uses addres
& checks whether it gets an a

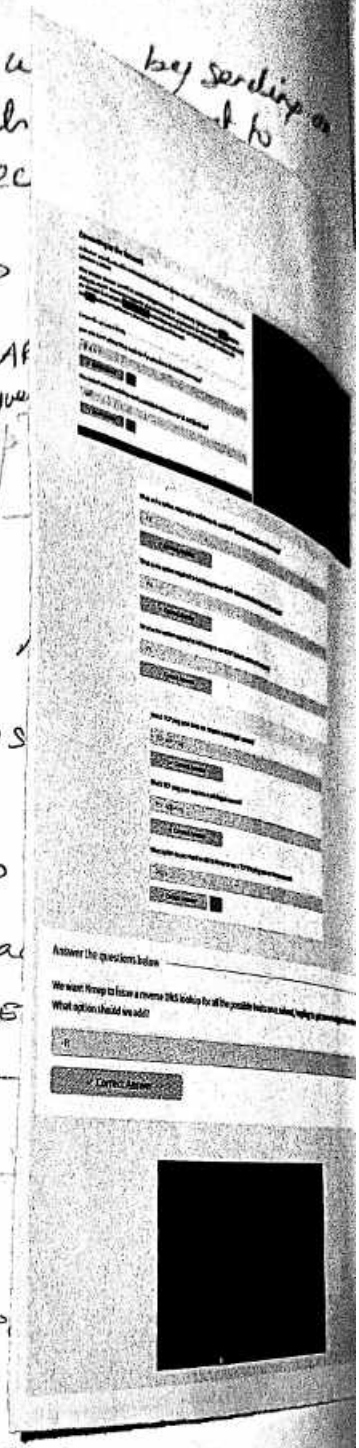
nmap -PM -sn TARGE



NMAP HOST Discovery using T
-TCP SYN Ping.

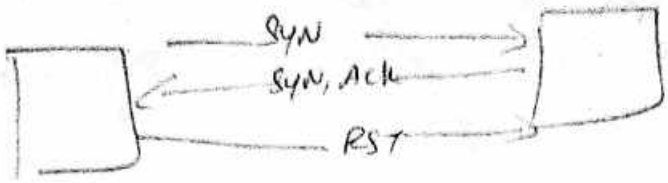


TCP port is open



Privileged users can send TCP SYN packets without completing the 3-way handshake

nmap -PS -sn -TARGET

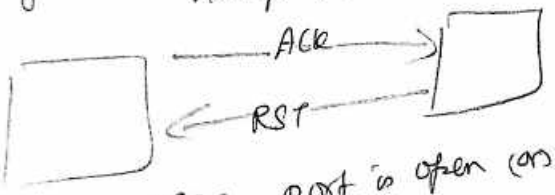


TCP port is open

TCP ACK PING

To utilize ACK ping in Nmap which sends a packet with ACK flag set, then Nmap runs as a privileged user.

nmap -PA -sn TARGET

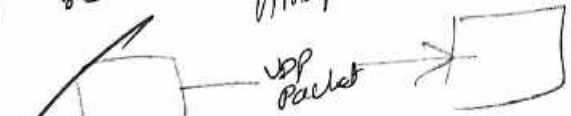


TCP port is open (or) closed.

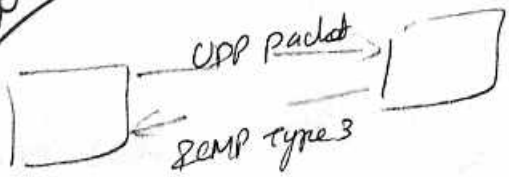
UDP Ping:

Unlike TCP ping, sending a UDP packet to an open port does not elicit a response. When sending a UDP packet to a closed UDP port, you anticipate receiving an ICMP port unreachable packet which indicates the target system is active and reachable.

nmap -PU -sn -TARGET



UDP port is open



UDP port is closed

RESULT: Thus the above nmap scan tool is used to learn how ARP, ICMP, TCP and UDP can detect live hosts by completing this session.

Ex NO:9

IMPLEMENTATION OF SUBNETTING IN CISCO PACKET TRACER SIMULATOR

AIM:

Implementation of subnetting in CISCO packet tracer Simulator.

INTRODUCTION:

Classless IP subnetting is a technique that allows for more efficient use of IP addresses by allowing for subnet mask that are not just the default mask for each IP class.

CREATING A NETWORK TOPOLOGY

The first step in implementing classless IP subnetting is to create a network topology in packet tracer. In packet tracer, select the "new" button in the top left corner. Then select "Network" and "Generic". This will create a blank network topology that we can use to add devices.

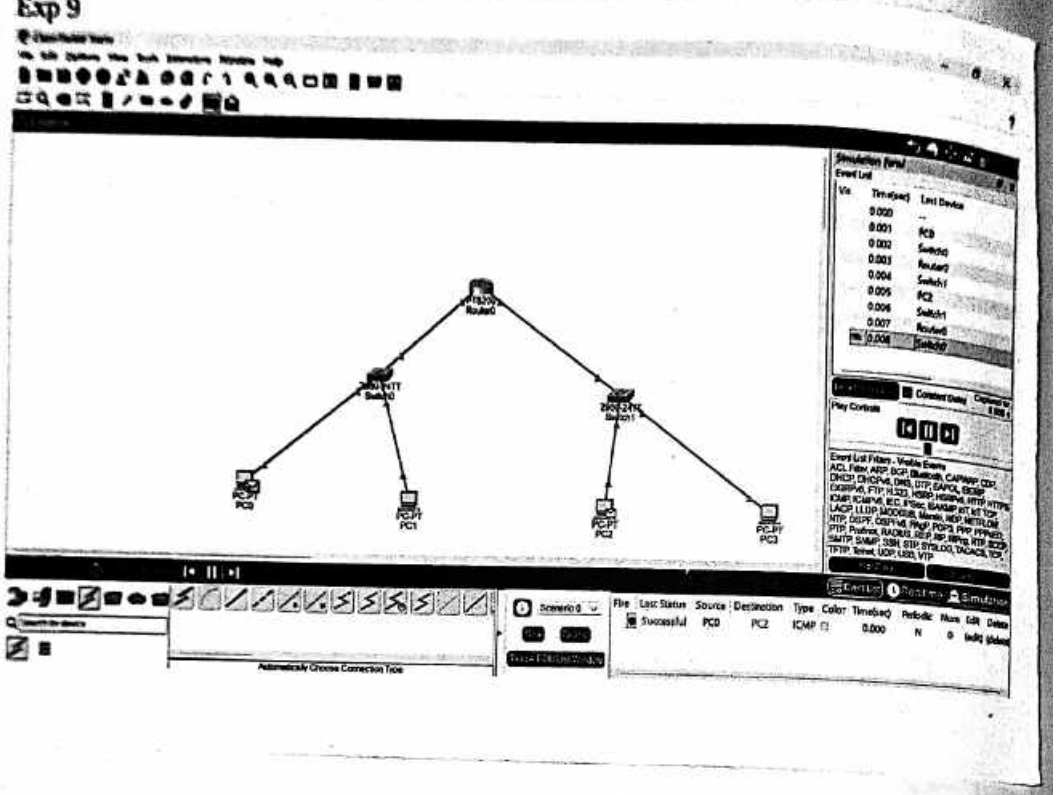
ADDING THE DEVICES:

Once we have created our network topology, we can add devices to it, we will be adding routers, switches and PCs.

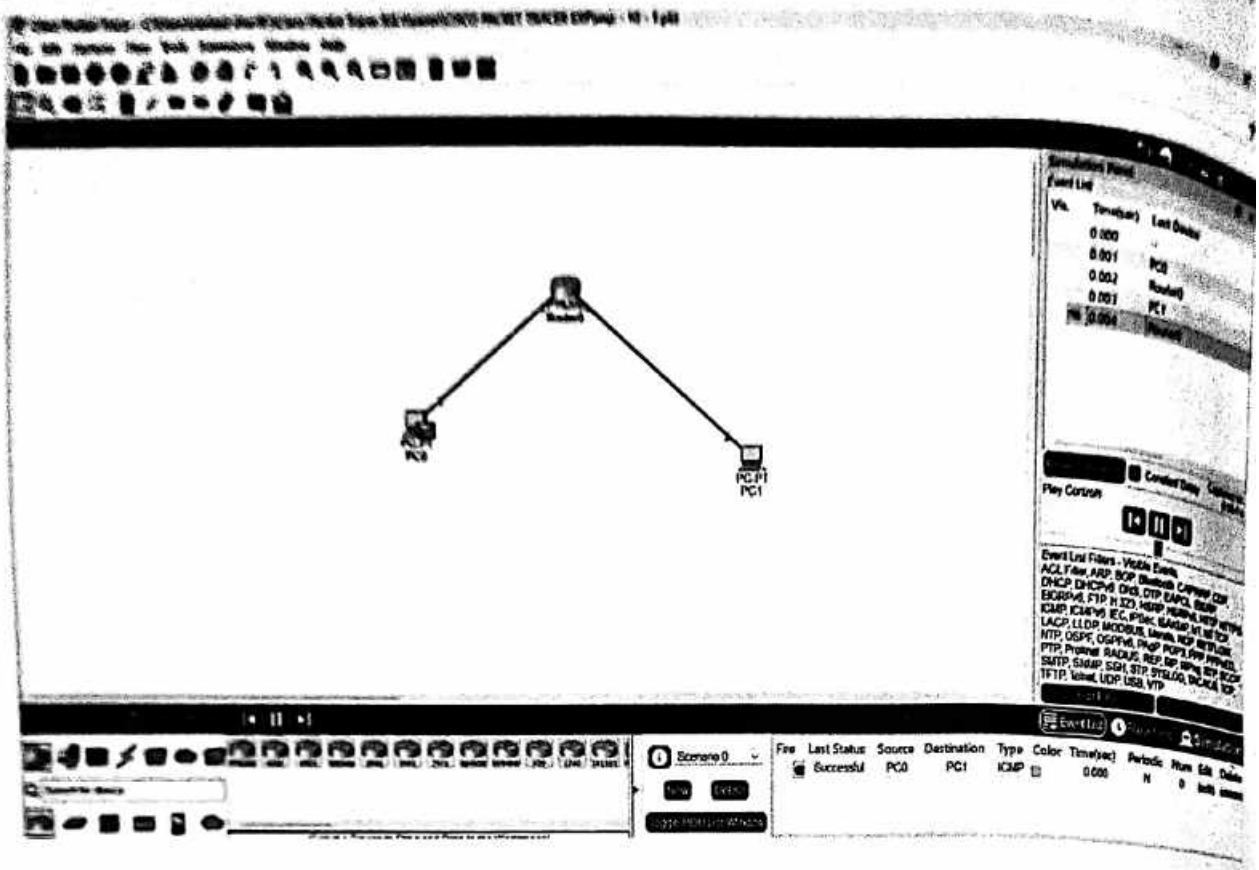
SUBNETTING

To subnet the network address of 192.168.1.0/24 to provide enough space for at least 5 addresses for end devices, the switch and the router. we can use a /27 subnet mask. This will give us 8 subnets with 30 host addresses each.

Exp 9


Router0
Switch0
Switch1
PC0
PC1
PC2
PC3
Simulation Panel
Event List
Play Controls
Scenario 0
Automatically Choose Connection Type
Fire
Last Status
Source
Destination
Type
Color
Time(sec)
Periodic
Num
Edit
Delete
Successful
PC0
PC2
ICMP
0.000
N
0

Expt 10


PC0
PC1
Scenario 0
Fire
Last Status
Source
Destination
Type
Color
Time(sec)
Periodic
Successful
PC0
PC1
ICMP
0.000

EX NO: 10 a) INTERNETWORKING WITH ROUTERS IN CISCO PACKET TRACER SIMULATOR

AIM:

To implement the internetworking with Routers in CISCO Packet tracer simulator.

INTRODUCTION:

In this network, a router and 2 PCs are used. Computers are connected with routers using a copper straight-through cable. After forming the network to check network connectivity a simple PDU is transferred from PC0 to PC1.

PROCEDURE:

Step 1:

configuring Router 0

1. Select the router and open CLI
2. Press ENTER to start configuring Router 1
3. Type enable to activate the privileged mode.

Step 2: configuring PCs.

1. Assign IP addresses to every PC in the n/w.
2. Select the PC, Go to the desktop and select IP Configuration and assign an IP address. Default gateway, subnet mask.
3. Assign the default gateway of PC0 as 192.168.10.1
4. Assign the default gateway of PC1 as 192.168.20.1

Step 3: connecting PCs with Router.

1. Connect Fast Ethernet 0 port of PC0 with Fast Ethernet 0/0 port of Router1 using a copper Straight thro cable.

4) INTERNETWORKING USING WIRELESS ROUTER, DHCP SERVER AND INTERNET CLOUD.

AIM

To design and configure an internetworking using wireless router, DHCP server and internet cloud.

OBJECTIVES:

Part 1: Build a simple network in the logical topology workspace.

Part 2: Configure the Network Devices

Part 3: Test connectivity between Devices

Part 4: Save the file & close packet tracer.

Part 1: Build a simple N/w in the logical topology workspace.

Step 1: Launch packet tracer.

Step 2: Build the topology

a. Add network devices to the workspace.

b. Change display names of the network devices

c. Add the physical cabling between devices on the workspace.

Part 2: Configure the network Device

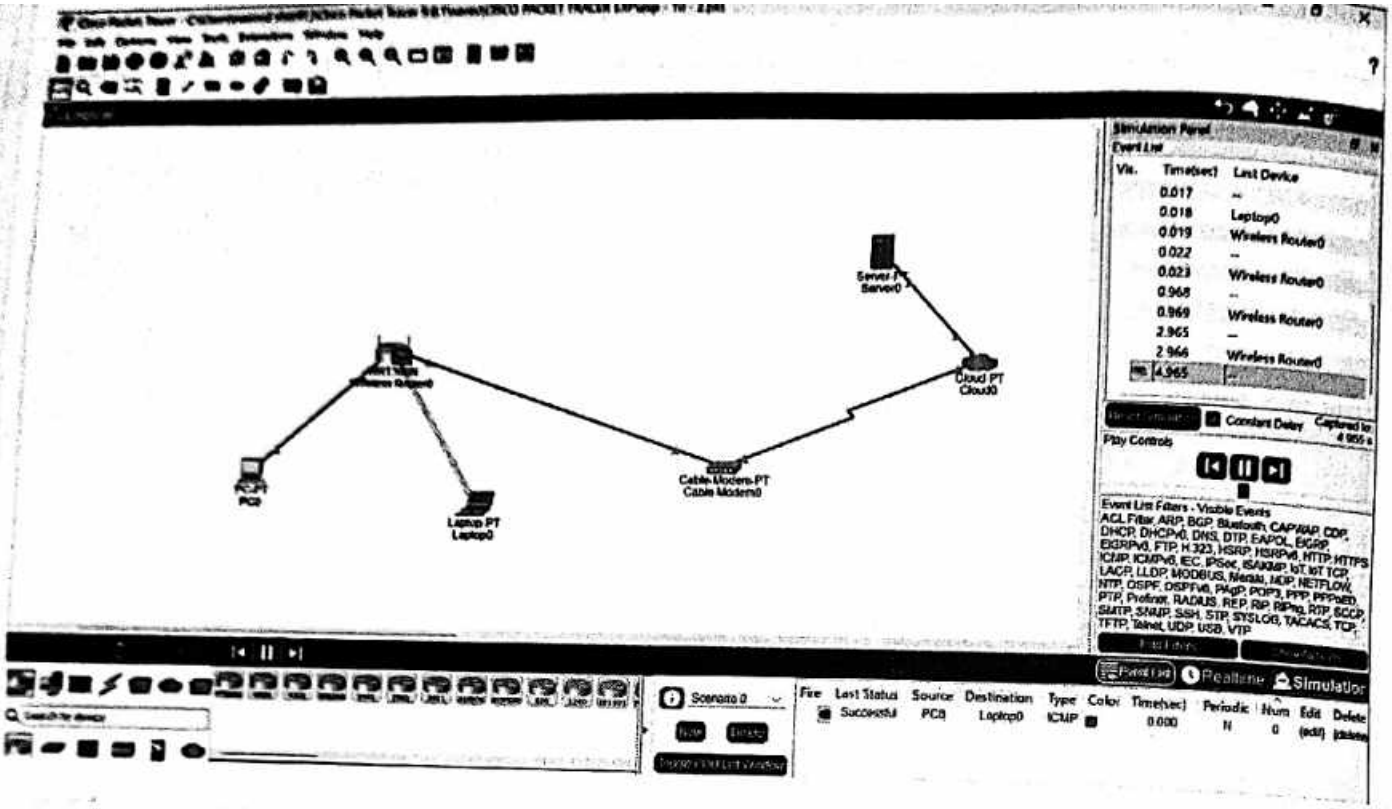
Step 1: Configure the wireless router

a. Create the wireless n/w on the wireless router.

b. Click on the save settings tab

Step 2: Configure the Laptop

Configure the laptop to access the


Server PT
Server0
Cloud PT
Cloud0
Cable Modem-PT
Cable Modem0
PC0
Laptop PT
Laptop0
Simulation Panel
Event List
Vis. Time(sec) Last Device
0.017 --
0.018 Laptop0
0.019 Wireless Router0
0.022 --
0.023 Wireless Router0
0.968 --
0.969 Wireless Router0
2.965 --
2.966 Wireless Router0
4.965 --
Play Controls
Event List Filters - Visible Events
Realtime
Simulation
Scenario 0
Fire Last Status Source Destination Type Color Time(sec) Periodic Num Edit Delete
Successful PC0 Laptop0 ICMP 0.000 N 0 (edit)

Step3: Configure the PC

Step4: Configure the Internet cloud.

a. Install network modules if necessary.

b. Identify the from & To ports

Step5: Configure the CISCO.com Server:

a. Configure the CISCO.com Server as a DHCP Server.

b. Configure the CISCO.com Server as a DNS Server to provide domain name to IPV4 address resolution.

c. Configure the CISCO.com Server cloud

d. Configure the CISCO.com Server FastEthernet() Interface settings.

Part3: Verify Connectivity.

Step1: Refresh the IPV4 settings on the PC

a. Verify that the PC is receiving IPV4 configuration information from DHCP

b. Test connectivity to the CISCO.com Server from the PC.

RESULT:

Thus the Interwork using wireless router, DHCP server and Internet cloud has been designed and configured successfully

Ex No:11

ROUTING AT NETWORK LAYER.

a) STATIC ROUTING CONFIGURATION USING CISCO PACKET TRACER.

AIM:

To Simulate Static Routing configuration using CISCO Packet Tracer.

INTRODUCTION:

Static routes are the routes given which are manually added to the router's routing table. The process of adding static routes to the routing table is known as static routing.

Setting up a practice lab:

1. Create a packet tracer lab. with initial IP configuration
2. Creating, adding verifying static routes

Routes requirements:

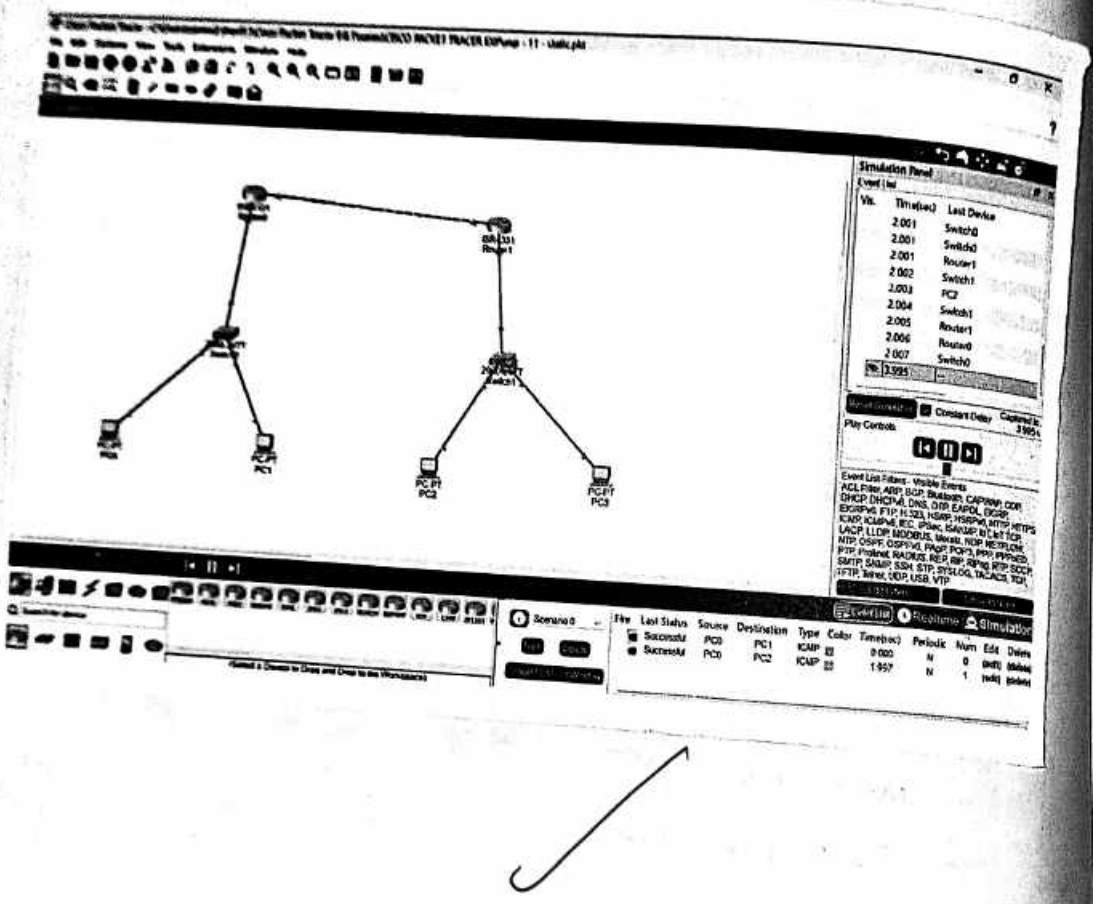
1. Create three routes for network and configure the first route as the main route and the second route as a backup route.
2. Create two routes for host. Configure first route as main route and the second route as a backup route.
3. Verify the routes & adds only main route to the routing table.

Routes configuration

Verifying Static routing:

* By sending ping request to a PC of n/w and tracing the path they take to reach the n/w.

expt 11


Simulation Panel
Time(sec)
Last Device
Switch0
Switch0
Router1
Switch1
PC2
Switch1
Router1
Router0
Switch0
Play Controls
Event List Filters - Visible Events
Realtime
Simulation
Fire
Last Status
Source
Destination
Type
Color
Time(sec)
Periodic
Num
Edit
Delete
Successful
PC0
PC1
ICMP
0.000
N
0
Successful
PC0
PC2
ICMP
1.957
N
1

- For this, 'traceroute' command on a PC then is used
- The 'Traceroute' command sends ping request to the destination host and finds the path they take to reach the destination.

RESULT:

Then the static routing is configured and tested using CISCO packet tracer.

(b) RIP USING CISCO PACKET TRACER

AIM:

To simulate RIP using CISCO Packet tracer.

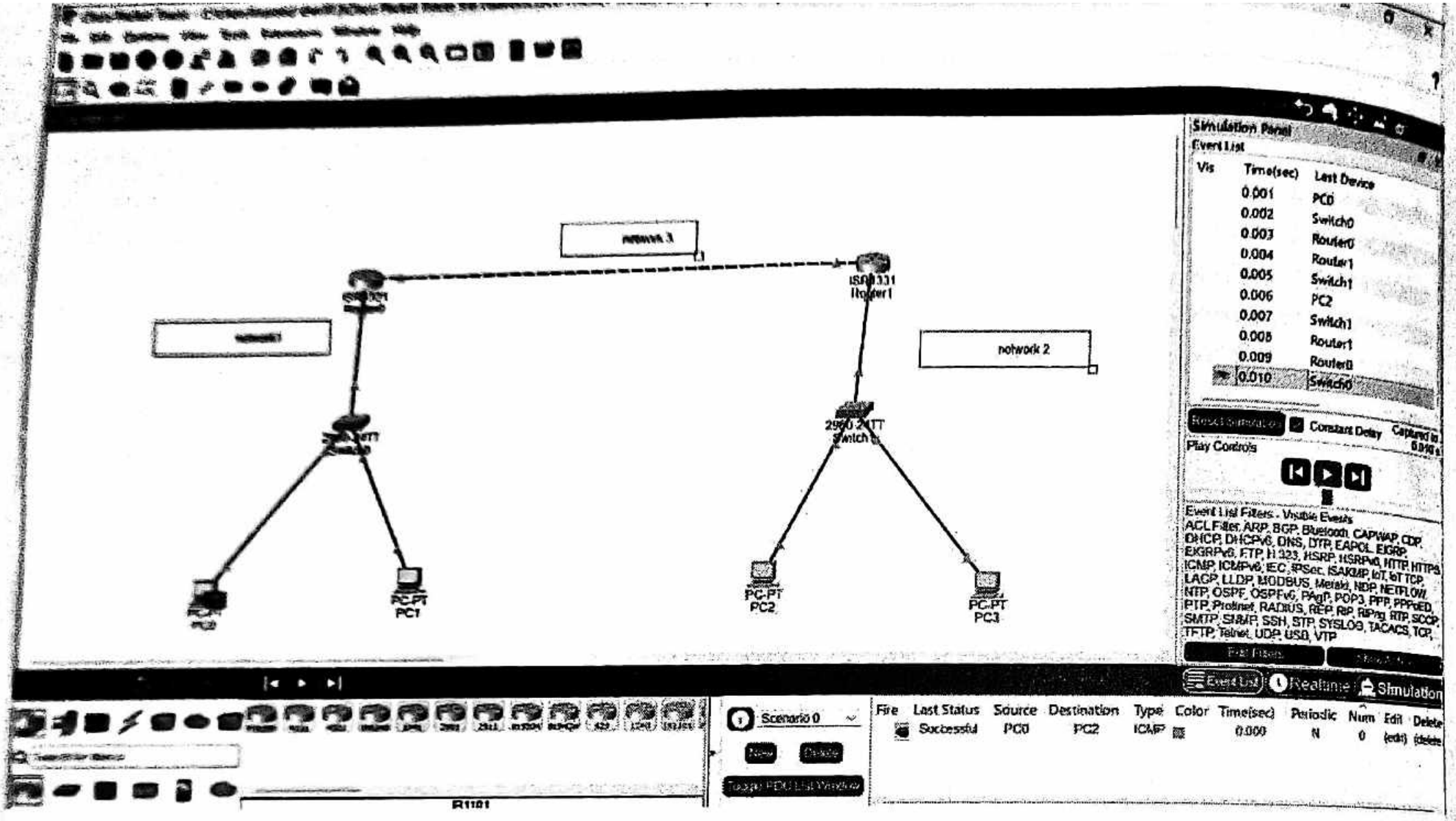
PROCEDURE:

1. Assign IP address to PCs.
2. Assign IP address to Interface of routers

Configure RIP Routing protocol.

1. Enable RIP Routing protocol from global configuration mode.
2. Tell RIP routing protocol which networks you want to advertise.
3. RIP protocol automatically manages all routes
4. If one route goes down. it automatically switches to another available.

RESULT: Thus the Simulation of RIP using CISCO packet tracer has been successfully done.


network 3
network 2
PC-PT
PC1
PC-PT
PC2
PC-PT
PC3
Simulation Panel
Event List
Vis
Time(sec)
Last Device
0.001
PC0
0.002
Switch0
0.003
Router0
0.004
Router1
0.005
Switch1
0.006
PC2
0.007
Switch1
0.008
Router1
0.009
Router0
0.010
Switch0
Constant Delay
Play Controls
Event List Filters - Visible Events
ACL Filter, ARP, BGP, Bluetooth, CAPWAP, CDP, DHCP, DHCPv6, DNS, DTP, EAPOL, EIGRP, EIGRPv6, FTP, H.323, HSRP, HSRPv6, HTTP, HTTPS, ICMP, ICMPv6, IEC, IPSec, ISAKMP, IoT, IoT TCP, LACP, LLDP, MODBUS, Meraki, NDP, NETFLOW, NTP, OSPF, OSPFv6, PAgP, POP3, PPP, PPPoED, PTP, Prolinet, RADIUS, REP, RIP, RIPng, RTP, SCCP, SMTP, SNMP, SSH, STP, SYSLOG, TACACS, TCP, TFTP, Telnet, UDP, USB, VTP
Edit Filters
Show All/None
Event List
Realtime
Simulation
Scenario 0
New
Delete
Toggle PDU List Window
Fire
Last Status
Source
Destination
Type
Color
Time(sec)
Periodic
Num
Edit
Delete
Successful
PC0
PC2
ICMP
0.000
N
0
(edit)
(delete)

Ex.No.12 END TO END COMMUNICATION AT TRANSPORT LAYER

(a) Implement ECHO client server using TCP sockets

AIM:

To implement an ECHO SERVER application using TCP sockets.

CODE:

ECHO SERVER

```
import socket
S = socket.socket()
S.bind(("localhost", 5000))
s.listen(1)
C, addr = S.accept()
msg = C.recv(1024).decode()
print("client :", msg)
C.send(msg.encode())
C.close()
S.close()
```

ECHO CLIENT

```
import socket
S = socket.socket()
S.connect(("localhost", 5000))
msg = input("Enter message:")
S.send(msg.encode())
print("Echo :", S.recv(1024).decode())
S.close()
```

Input:

Echo message : Hello server

output.

SERVER

client connected
Received : Hello server

CLIENT

Echo from server: Hello server.

8/11

Result:

The ECHO client server using TCP socket was studied and executed successfully.

(b) Implement chat client server using TCP/IP sockets

AIM:

To implement a chat client server application using TCP sockets.

CODE:

SERVER

```
import socket
s= socket.socket()
s.bind(("localhost", 5000))
s.listen(1)
c, a = s.accept()
while True:
    msg = c.recv(1024).decode()
    print("client : "msg)
    if msg == "byte":
        break
    reply = input("server : ")
    c.send(reply.encode())
c.close()
```

CLIENT

```
import socket
s = socket.socket()
s.connect(("localhost", 5000))
while True:
    msg = input("client :")
    s.send(msg.encode())
```

```
    if msg == "bye"
        break
    print("server: ", s.recv(1024).decode())
s.close()
```

Input:

Client : Hello

server : Hi

Client : bye

Output:

Client : Hello

server : Hi

RESULT:

The chat client server application were succesfully studied and implemented using TCP sockets.

Ex. No. 13

IMPLEMENT YOUR OWN PING PROGRAM

AIM:

To implement a ping program using PYTHON to check the connectivity between source and destination host.

ALGORITHM

1. Get the destination IP address from user.
2. Send the request to destination
3. Wait for response.
4. Calculate the response time.
5. Display whether the host is reachable.
6. Repeat the proces for few request.

CODE:

```
import os
import time
host=(input(" Enter host : ")
for i in range (4),
    start = time.time()
    result = os.system(" ping -n 1 " + host + ">null")
    end = time.time()
    if result ==0:
        print (" Reply from", host,
        " Time =", round ((end - start) * 1000, 2),
                                        "ms")
    else
        print (" Request timed out")
    time.sleep(1)
```

Input:

Enter host : 127.0.0.1

output.

Reply from 127.0.0.1 Time = 1.23 ms

Reply from 127.0.0.1 Time = 0.98 ms

RESULT:

The ping program was successfully studied and Implemented.

EX.NO: 14

PACKET SNIFFING USING RAW SOCKET

AIM:

To write a program using RAW sockets to capture and display network packets

ALGORITHM:

1. Create a RAW Socket
2. Bind the socket to the network interface
3. Receive packets from the network
4. Extract the IP header information
5. Display the source IP, destination IP and protocol.
6. Continue capturing packets

CODE:

```
import socket

s = socket.socket(socket.AF_INET, socket.
    SOCK_RAW, socket.IPPROTO_IP)

s.bind(("0.0.0.0",0))

s.setsockopt(socket.IPPROTO_IP,socket.IP_
                                HDRINCL,1)

print("Packet Sniffer started...")

while True:

    data, addr = s.recvfrom(65535)

    print("Packet received from:", addr[0])
```

Input:

Network Packets

Output:

packet Sniffer started:
packet received from: 192.168.1.5
packet received from: 192.168.1.1
packet received from: 192.180.184.14

RESULT:

The RAW SOCKET SNIFFING PROGRAM was successfully implemented and network packets was captured and displayed.